

10.实验报告:

CNN 训练过程分析、优化器影响与内部特征可视化

一、实验目的

- 1.理解优化器和学习率对 CNN 训练过程的影响。
- 2.能够比较 SGD、Momentum 和 Adam 优化器的训练效果。
- 3.可视化 CNN 的卷积核和特征图 (featuremaps)。
- 4.分析模型的错误分类样本, 绘制混淆矩阵。
- 5.使用曲线和混淆矩阵评价模型表现。

二、实验环境

操作系统: Windows11+WSL2(Ubuntu22.04)

深度学习框架: PyTorch2.12.0

硬件: CPU (GPU 驱动版本过低, 自动回退 CPU)

数据集: MNIST 手写数字 (28×28 灰度图, 10 类)

三、实验方法及模型结构

CNN 模型

```
```python
SimpleCNN(
(conv1):Conv2d(1,32,kernel_size=3,padding=1)
(conv2):Conv2d(32,64,kernel_size=3,padding=1)
(pool):MaxPool2d(2,2)
(fc1):Linear(3136,128)
(fc2):Linear(128,10)
(dropout):Dropout(0.25)
)
```
```

数据划分:

训练集: 48,000 张 (80%)

验证集: 12,000 张 (20%)

测试集: 10,000 张

四、实验任务与结果

任务 1: 使用上次 CNN 模型

使用相同的模型结构 Adam 优化器 (lr=0.001) 重新训练 5 个 epoch, 得到基准模型。测试集准确率为 98.90%, 证明模型有效, 可用于可视化和错误分析。

任务 2: 优化器对比

使用相同模型和数据集, 分别训练 SGD (lr=0.01)、SGD+Momentum (lr=0.01,momentum=0.9) 和 Adam (lr=0.001), 每个训练 5 个 epoch。记录每个 epoch 的训练/验证损失、准确率, 以及最终测试准确率。

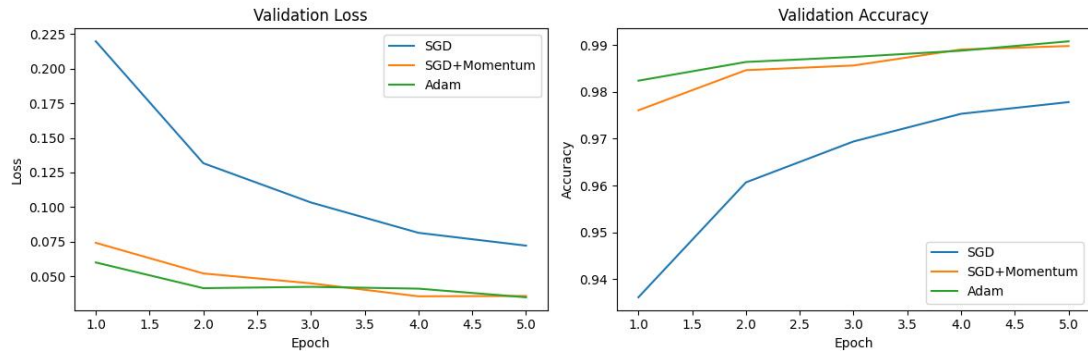
训练过程结果:

| 优化器 | Epoch1ValA
cc | Epoch2ValA
cc | Epoch3ValA
cc | Epoch4ValA
cc | Epoch5ValA
cc | 测试准
确率 |
|------------------|------------------|------------------|------------------|------------------|------------------|-----------|
| SGD | 93.61% | 96.07% | 96.94% | 97.53% | 97.78% | 97.92% |
| SGD+Moment
um | 97.61% | 98.47% | 98.57% | 98.91% | 98.98% | 99.05% |
| Adam | 98.24% | 98.64% | 98.75% | 98.88% | 99.08% | 99.14% |

分析：

验证损失和准确率曲线（见下图）：

生成图片：`optimizer\_comparison.png`



Adam 收敛最快，第一个 epoch 验证准确率已达 98.24%，最终测试准确率最高（99.14%）。SGD+Momentum 收敛速度次之，最终测试准确率 99.05%，与 Adam 非常接近，且比纯 SGD 稳定很多。

纯 SGD 收敛最慢，最终测试准确率 97.92%，明显低于前两者。

结论：对于 MNIST 这类简单任务，Adam 和带动量的 SGD 都能取得优秀结果；Adam 无需调整学习率，SGD+Momentum 在适当学习率下也能达到接近的性能。

任务 3：学习率对比

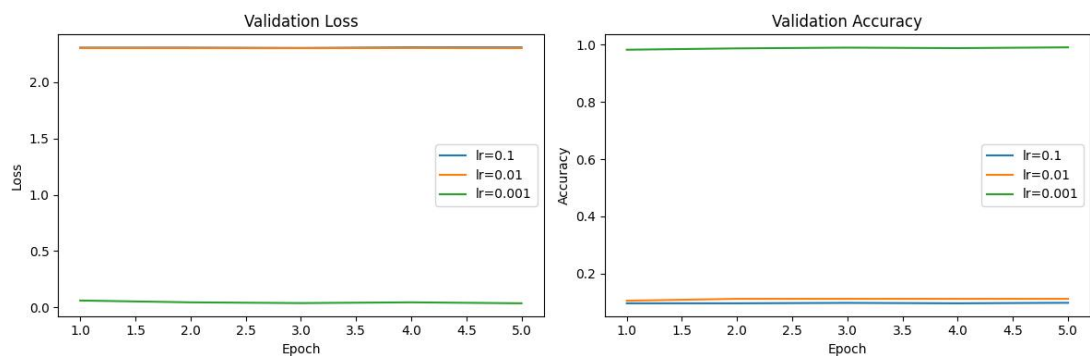
固定 Adam 优化器，分别尝试 lr=0.1,0.01,0.001。训练 5 个 epoch。

| 学习率 | 训练损失（最终） | 验证损失（最终） | 验证准确率 | 现象 |
|-------|----------|----------|--------|----------------------|
| 0.1 | 2.3104 | 2.3086 | 9.74% | 损失不下降，准确率约 10%（随机猜测） |
| 0.01 | 2.3022 | 2.3017 | 11.35% | 同样不收敛，准确率极低 |
| 0.001 | 0.0274 | 0.0357 | 99.01% | 正常收敛，准确率高 |

分析：

验证损失和准确率曲线（见下图）：

生成图片：`lr\_comparison.png`



学习率 0.1 和 0.01 过大，导致损失震荡甚至发散，模型无法学习（准确率接近 10%，相当于随机猜测）。

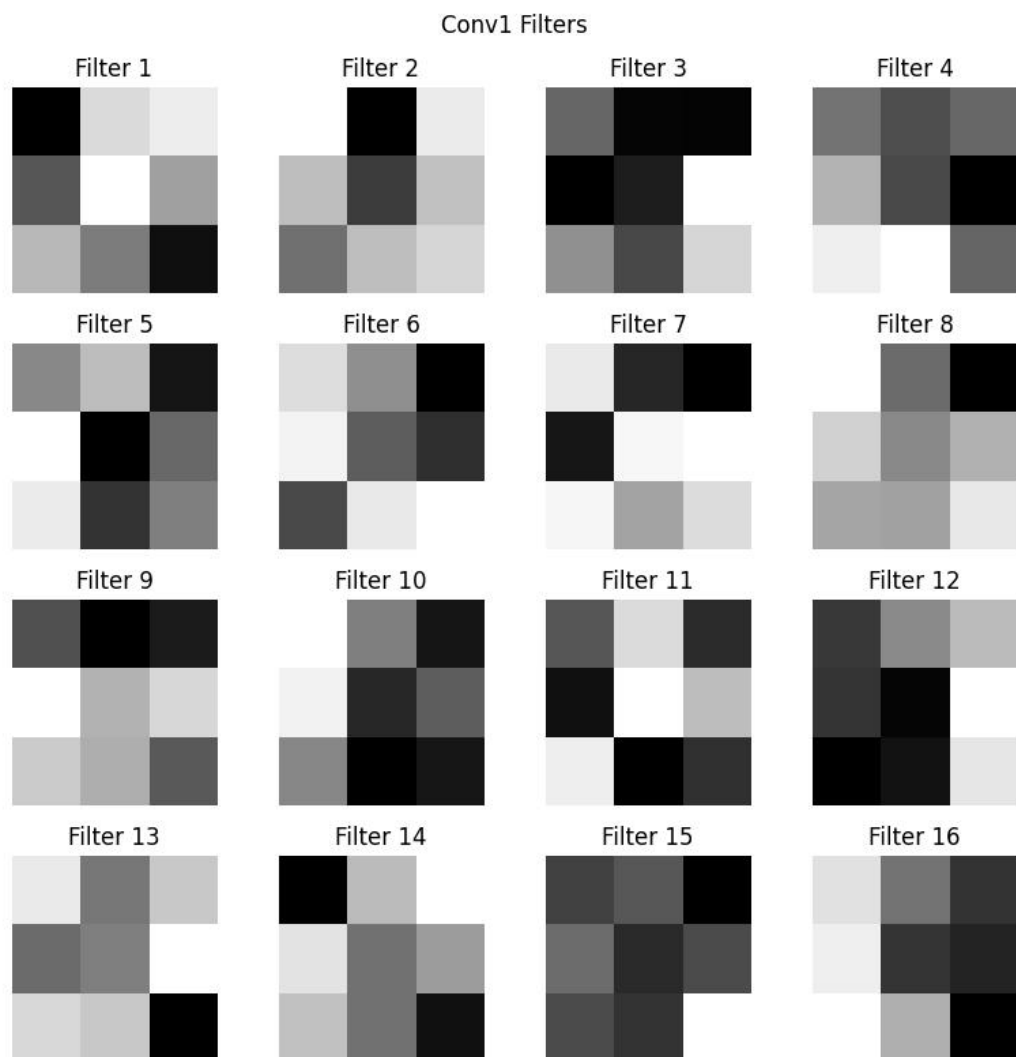
学习率 0.001 是合适的，损失稳步下降，准确率快速上升至 99% 以上。

结论：学习率是影响训练成败的关键超参数。过大会导致不收敛，过小会收敛缓慢。对于 Adam 优化器，建议从 0.001 开始尝试。

任务 4：卷积核可视化

结果：下图展示了第一层卷积层的 16 个卷积核（大小 3×3 ）。

生成图片：`conv1\_filters.png`



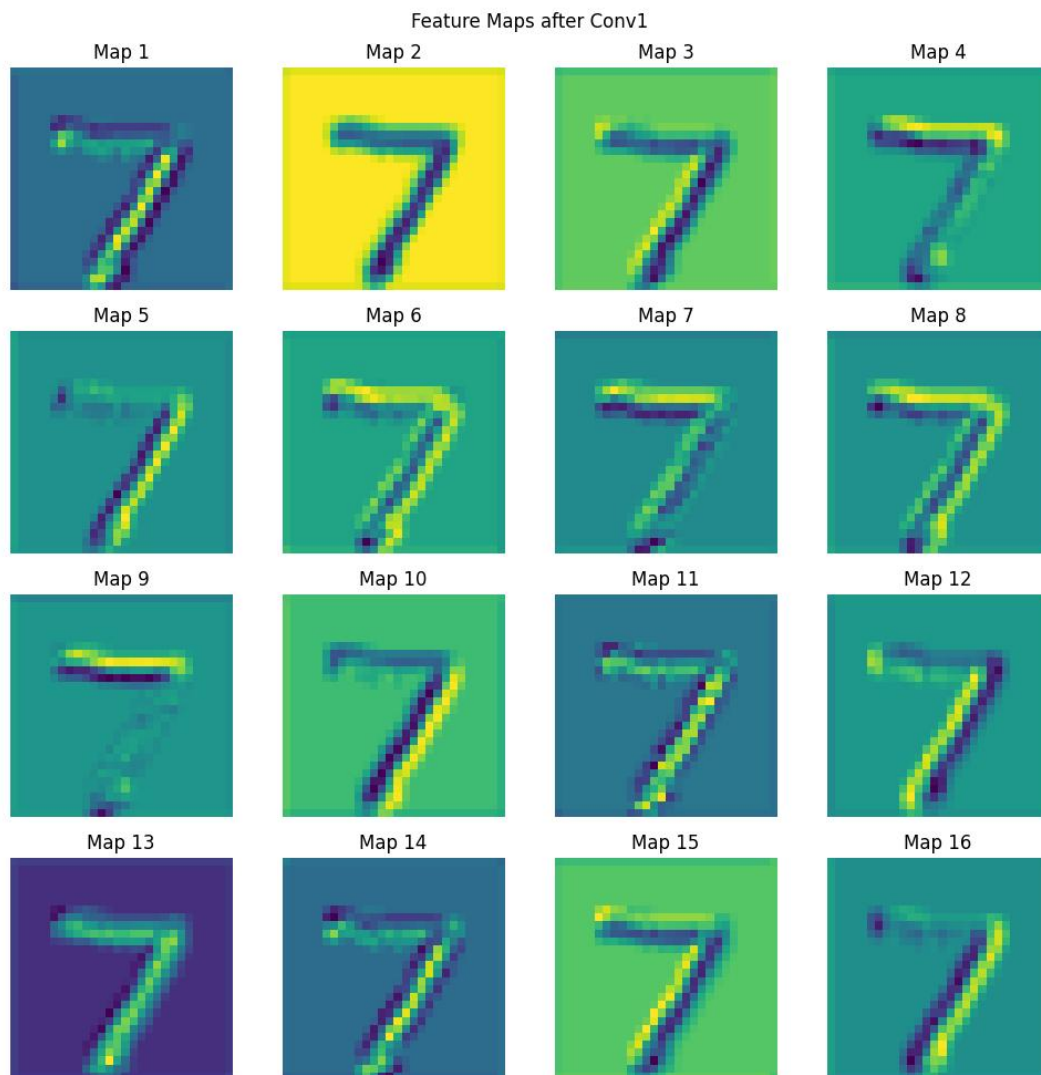
分析：

训练后的卷积核呈现出边缘检测、方向滤波器（水平、垂直、斜向）以及纹理模式。这些特征不是手工设计的，而是通过反向传播从数据中自动学习得到的。不同卷积核负责检测不同的局部模式，为后续特征提取奠定基础。

任务 5：Featuremap 可视化

方法：选择一张测试图像（数字 0），输入网络，提取第一层卷积输出的特征图（32 个通道）。下图展示了前 16 个特征图。

生成图片：`feature\_maps.png`



观察：

不同特征图对图像的不同区域有强响应：有的对边缘敏感，有的对笔画的粗细敏感，有的对背景与前景的对比敏感。

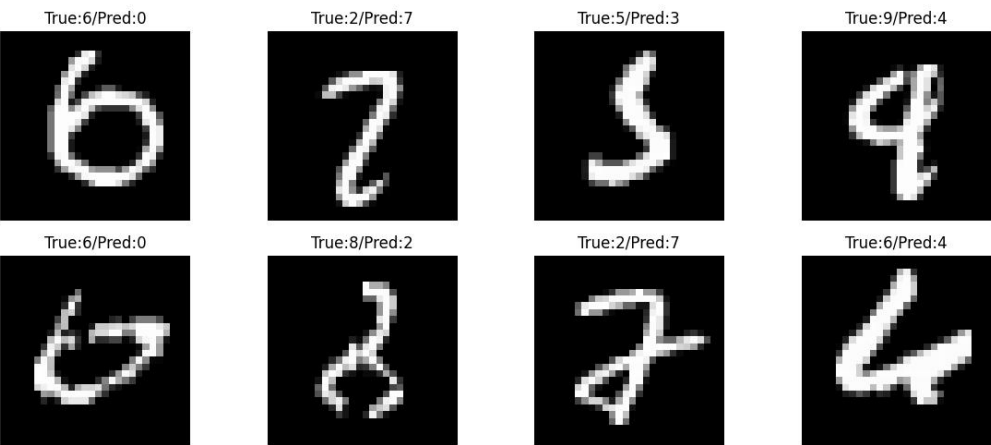
例如，某些特征图高亮数字的上半部分，另一些高亮下半部分。

这表明卷积层能够自动学习到多样化的局部特征，这是 CNN 强大的特征提取能力的体现。

任务 6：错误分类样本分析

结果：从测试集中找出模型预测错误的样本，显示 8 张图片（见下图）。

生成图片：`error\_samples.png`



混淆统计（基于实际运行结果）：

6→0:2 次
2→7:2 次
5→3:1 次
9→4:1 次
8→2:1 次

分析：

最易混淆的数字：6 和 0、2 和 7、9 和 4、5 和 3、8 和 2。

错误原因：

形状相似：如 6 和 0 都包含封闭圆圈；2 和 7 在手写体中倾斜角度相似；9 和 4 尾部弯曲类似。

书写风格差异：某些数字写得太潦草或笔画过细。

图像噪声或模糊导致关键特征丢失。

改进建议：

数据增强：旋转、缩放、平移、弹性变形，增加样本多样性。

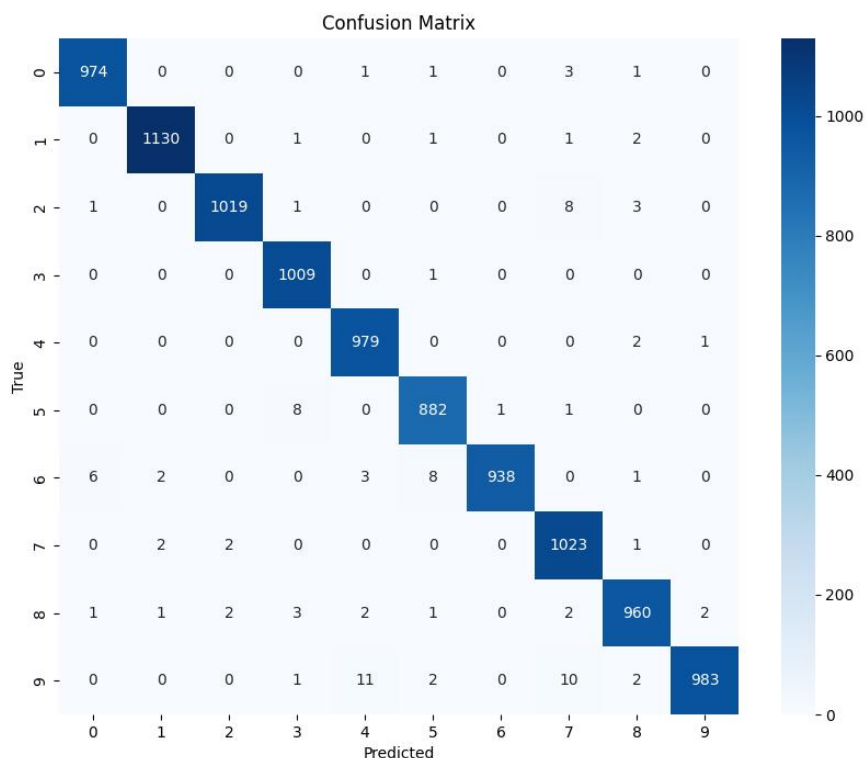
模型结构：增加卷积层深度或宽度，引入 BatchNorm、Dropout 等正则化。

训练方法：使用学习率衰减、更长的训练轮数、集成多个模型。

任务 7：混淆矩阵

结果：绘制测试集上 10 个类别的混淆矩阵（见下图）。

插入图片：`confusion\_matrix.png`



分析：

对角线元素：表示正确分类的样本数，数值越大说明该类识别越准确。

非对角线元素：表示错误分类的样本数，例如第 9 行第 4 列的值表示真实数字 9 被预测为 4 的个数。

混淆最严重的类别对：9 和 4，错误次数为 11 次。其次是 6 和 0（9 次），2 和 7（8 次）。

这些混淆与人类视觉感知一致：9 和 4 在手写体中常常因为尾部相似而混淆。

五、实验总结

本次实验在之前 CNN 分类实验的基础上，深入分析了优化器、学习率对训练的影响，并可视化了卷积核和特征图，评估了错误分类样本。主要收获如下：

1. 优化器选择：Adam 收敛快、效果好，适合快速实验；SGD+Momentum 在精细调参后也能达到相近性能。
2. 学习率影响：学习率是关键超参数，过大不收敛，过小收敛慢，需要通过实验确定合适范围（MNIST 上 0.01 较好）。
3. 卷积核与特征图：通过可视化发现，CNN 自动学习到了边缘、方向、纹理等低级特征，这解释了其强大的特征提取能力。
4. 错误分析：通过混淆矩阵和错误样本展示，发现 9 与 4、6 与 0 等数字容易混淆，后续可通过数据增强或更复杂的模型来改进。
5. 整体流程：掌握了从训练、验证、测试到可视化分析的全流程，为更复杂的深度学习任务打下基础。